Team Project Specifications

Keep in Mind!!

- Programming is all about solving problem
- Code should be well designed and implemented following core principles for programming and software development

Team Project

- Teams (5-6 students)
- Already assigned in your respective lab group (check your xsite)
- No change requests for team change will be considered

What do you have to build?

- Each team picks **any real-world problem domain**
  - Find a real problem and build something that solves it



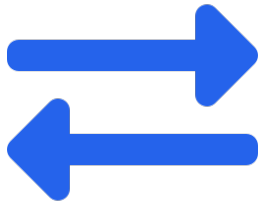
- The AI has to actually do the thinking, not just decorate the app
  - Your solution’s main purpose has to depend on it.



no AI, the logic won't run

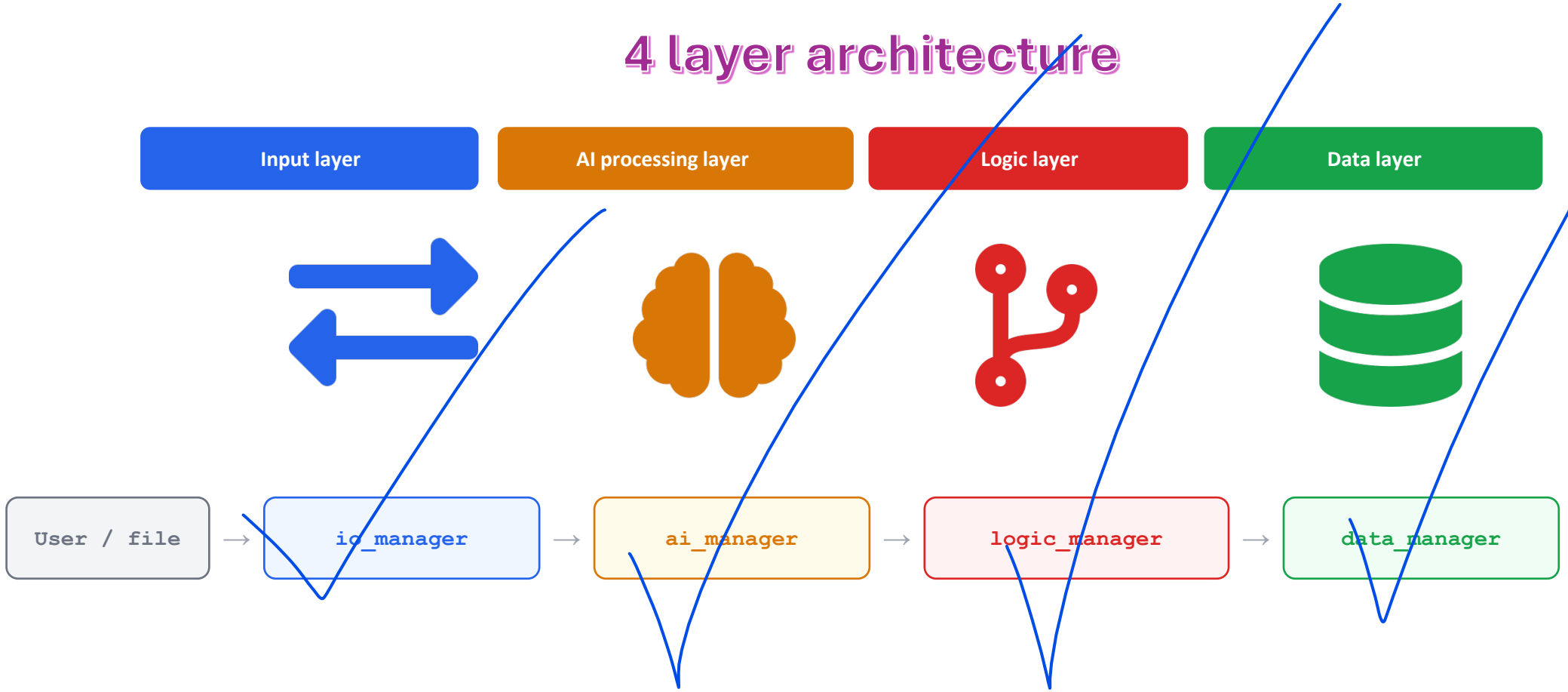
Your solution must contain

4 layer architecture



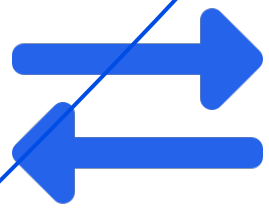



Your solution must contain



The Four Managers — Architecture Overview (1/4)

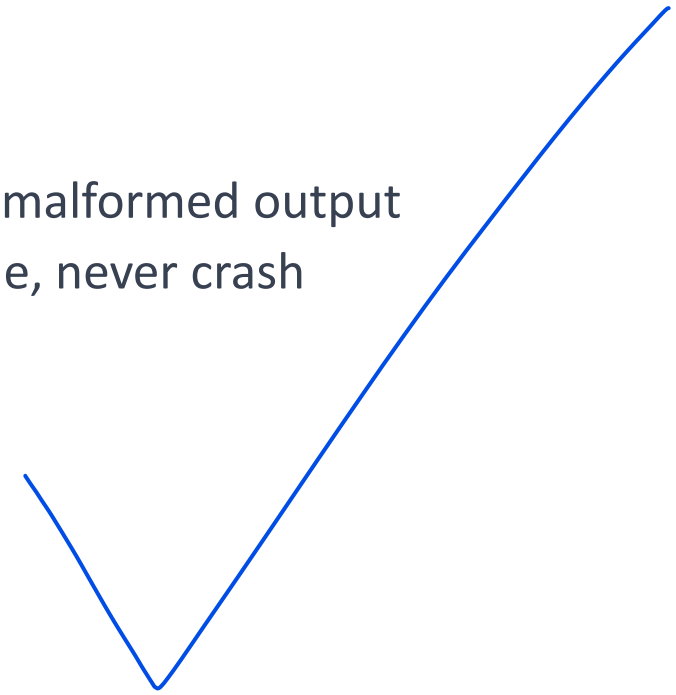
- I/O Manager
  - *All boundaries between system and user*
  - *Collect structured input from the user via the terminal.*
  - **Must implement**
    - Collect /summary views
    - and validate user input — reject and re-prompt on bad data
    - All print() calls in the system live here and nowhere else data_manager????
    - Format individual records and list



The Four Managers — Architecture Overview(2/4)

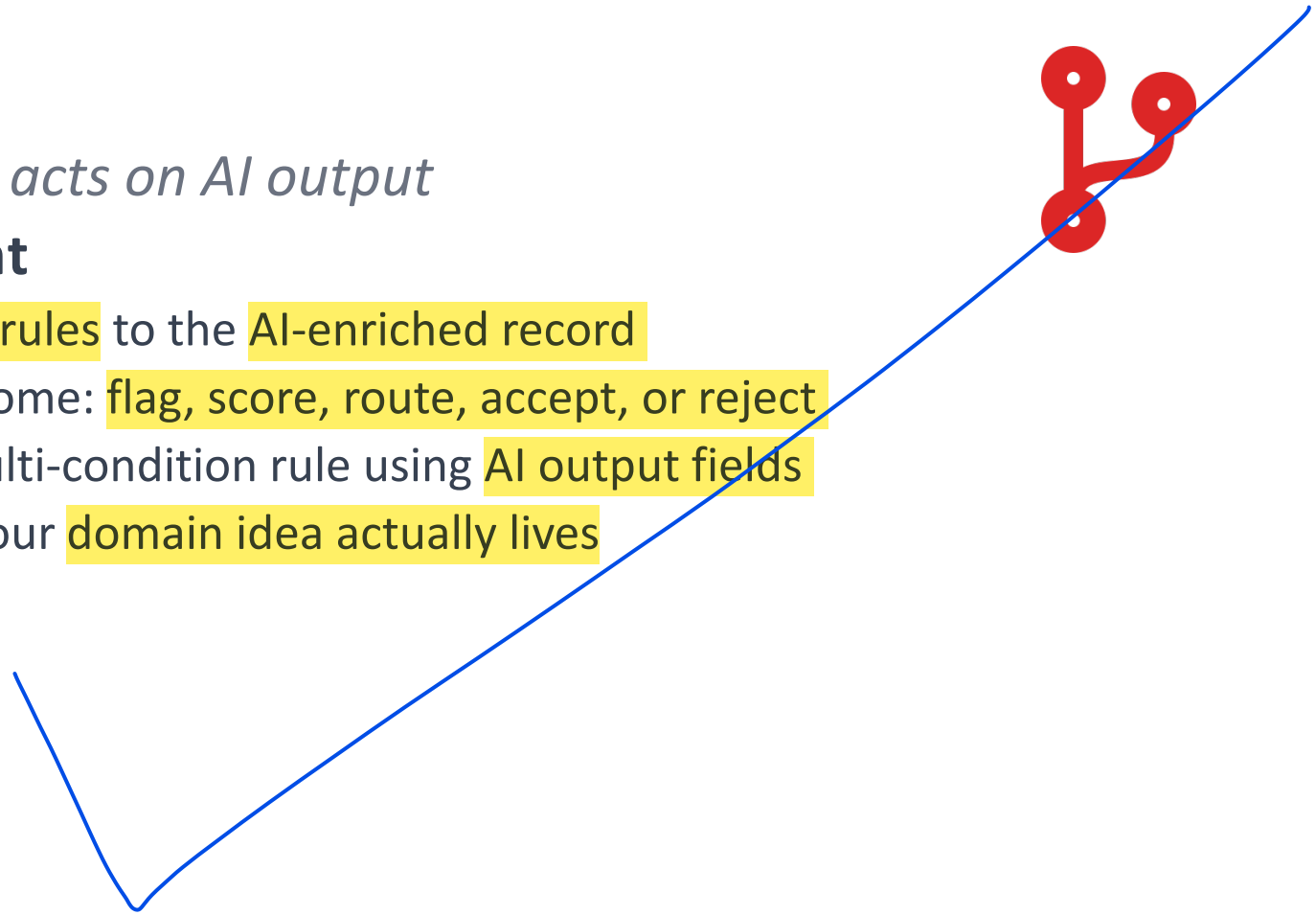
- AI Manager
  - *Core engine — every record passes through here*
  - *Every record passes through the AI API — no exceptions*
  - **Must implement**
    - Build a prompt from the input record
    - Call the API, parse response
    - Validate response schema — reject or retry on malformed output
    - Handle API failure gracefully — log and continue, never crash
    - Zero domain logic here — only API interaction





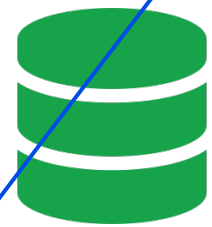
The Four Managers — Architecture Overview(3/4)

- Logic Manager
  - *Domain brain — acts on AI output*
  - **Must implement**
    - Apply business rules to the AI-enriched record
    - Decide an outcome: flag, score, route, accept, or reject
    - At least one multi-condition rule using AI output fields
    - This is where your domain idea actually lives



The Four Managers — Architecture Overview(4/4)

- Data Manager
  - *System memory across runs*
  - **Must implement**
    - Save processed records to CSV or JSON
    - Load all records on startup
    - At least one filter or query function
    - Handle missing or corrupt files without crashing



Example Domains (Not restricted)

domain selected: SME Grant Scheme Application

1. People & Human Services
2. Business & Operations
3. Safety, Risk and Compliance
4. Sustainability, Environment and Smart Communities
5. .....

Collaboration and Git

The Shared Sketchbook

- If Git is your code time machine, GitHub is the studio.
- Writing software individually is simple, but combining work with team members often leads to overwriting each other's changes.
- GitHub operates like Google Docs for code. It allows multiple developers to safely branch off, draft features asynchronously, comment on changes, and systematically merge them together once approved.

GitHub: Branching

1. VIEW & LIST

Check all available local branches and identify your current active workspace environment.

```
git branch
```

2. CREATE & SWITCH

Create a dedicated branch for developing a new feature or testing python scripts safely.

same as
git branch feature
git checkout feature

```
git checkout -b feature
```

3. PUBLISH BRANCH

Push your isolated feature branch to GitHub to open a pull request for review.

```
git push origin feature
```

Branching ensures your main production python branch remains clean and stable while you experiment.

we created several branches, 1 for each manager

Branching: Policies

1. Always start from an up-to-date base branch
2. Use descriptive and structured naming conventions
3. Keep branches focused and short-lived
4. Commit early and often locally
5. Push and sync branches regularly
6. Clean up and delete branches after merging

Hard Constraints — Non-Negotiable

1. **100% procedural** no class, no self, etc
   - No class definitions anywhere in the codebase. Not one. Assessors will grep. Functions only.
2. **AI is the core engine** AI_manager
   - Every record must pass through the AI API. If removing the API call doesn't break your app's main purpose, you've used AI wrong.
3. **Structured API responses**
   - The AI must return structured output (JSON). Your code must validate the schema before using it. Vague prompts that return unparseable text are a design failure.
4. **Flat file persistence** we selected JSON
   - All data saved to and loaded from CSV or JSON. The program must produce the same output across separate runs on the same input.
5. **Runs in Docker** make sure no "Works in my Machine" problem
   - Docker run on the submission branch must produce working output on any machine. The DevOps manager must verify this on all team laptops.
6. **Git history** but what constitutes a monolithic single commit dump? Do large dumps, but still non monolithic count?
   - Granular commits mapping to the development lifecycle. Monolithic single-commit dumps are penalised. We check the log

Project Check points

Week 2

Topic decided

- Finalize topic

Week 3

Speed Projecting

- Each team pitches their project idea to every other team in their lab group

Week 7

Submission

- Submission (code, report, test script, Docker, Git history)

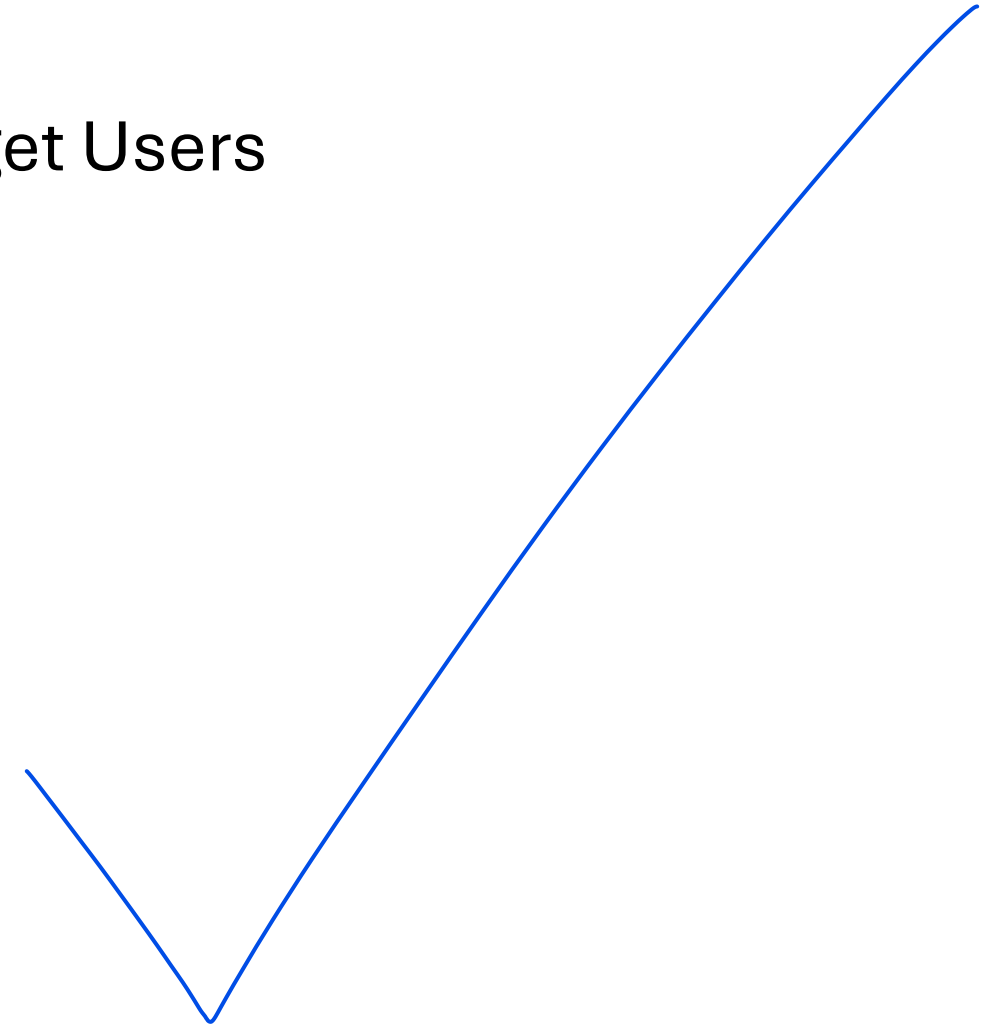
Week 8

Demo, Part 1

- Demo your submission

What needs to be submitted this week

1. Problem Statement and Target Users
2. User Inputs
3. Use of AI
4. Business Rules
5. Team Repository details



Remember

- Aim for Innovation/Complexity/Solving for the Problem
- Should focus on solving a problem better than other solutions to same problem (market value)
- Think whether proposed solution will work or not

Do aesthetics matter too?